

Task 12: Design and develop a RESTful API for Product Management using Spring Boot. Implement GET, POST, PUT, and DELETE methods and return JSON responses. Test the APIs using a REST client.

AIM:

To design and develop a RESTful API for Product Management using Spring Boot that performs CRUD operations (GET, POST, PUT, DELETE) and returns JSON responses. Also, to test the API using a REST client.

Algorithm:

1. Create Spring Boot project with required dependencies
2. Configure database connection
3. Create Product entity class
4. Create repository interface using JPA
5. Create service class for business logic
6. Create REST controller
7. Implement CRUD operations (GET, POST, PUT, DELETE)
8. Run the application
9. Test APIs using Postman

Program:

1. Product Entity:

```
import jakarta.persistence.*;

@Entity

public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private double price;

    private int quantity;

    public Product() {}

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }
```

```
public String getName() { return name; }  
public void setName(String name) { this.name = name; }  
public double getPrice() { return price; }  
public void setPrice(double price) { this.price = price; }  
public int getQuantity() { return quantity; }  
public void setQuantity(int quantity) { this.quantity = quantity; }  
}
```

2. Repository:

```
import org.springframework.data.jpa.repository.JpaRepository;  
public interface ProductRepository extends JpaRepository<Product, Long> {  
}
```

3. Service:

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;  
import java.util.List;  
  
@Service  
public class ProductService {  
    @Autowired  
    private ProductRepository repo;  
    public List<Product> getAllProducts() {  
        return repo.findAll();  
    }  
    public Product getProductById(Long id) {  
        return repo.findById(id).orElse(null);  
    }  
    public Product saveProduct(Product product) {  
        return repo.save(product);  
    }  
}
```

```

public Product updateProduct(Long id, Product product) {
    Product p = repo.findById(id).orElse(null);
    if (p != null) {
        p.setName(product.getName());
        p.setPrice(product.getPrice());
        p.setQuantity(product.getQuantity());
        return repo.save(p);
    }
    return null;
}

public void deleteProduct(Long id) {
    repo.deleteById(id);
}
}

```

4. Controller:

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductService service;

    @GetMapping
    public List<Product> getAllProducts() {
        return service.getAllProducts();
    }

    @GetMapping("/{id}")
    public Product getProduct(@PathVariable Long id) {

```

```

        return service.getProductById(id);
    }

    @PostMapping
    public Product createProduct(@RequestBody Product product) {
        return service.saveProduct(product);
    }

    @PutMapping("/{id}")
    public Product updateProduct(@PathVariable Long id, @RequestBody Product product) {
        return service.updateProduct(id, product);
    }

    @DeleteMapping("/{id}")
    public String deleteProduct(@PathVariable Long id) {
        service.deleteProduct(id);
        return "Deleted Successfully";
    }
}

```

Output:

```

{
  "id": 3,
  "name": "Headphones",
  "price": 1500.0,
  "quantity": 20
}

```

```

{
  "id": 1,
  "name": "Updated Laptop",
  "price": 55000.0,
  "quantity": 4
}

```

Result: The RESTful API for Product Management was successfully developed using Spring Boot. CRUD operations were implemented and tested using a REST client, confirming proper functionality and JSON-based communication.

